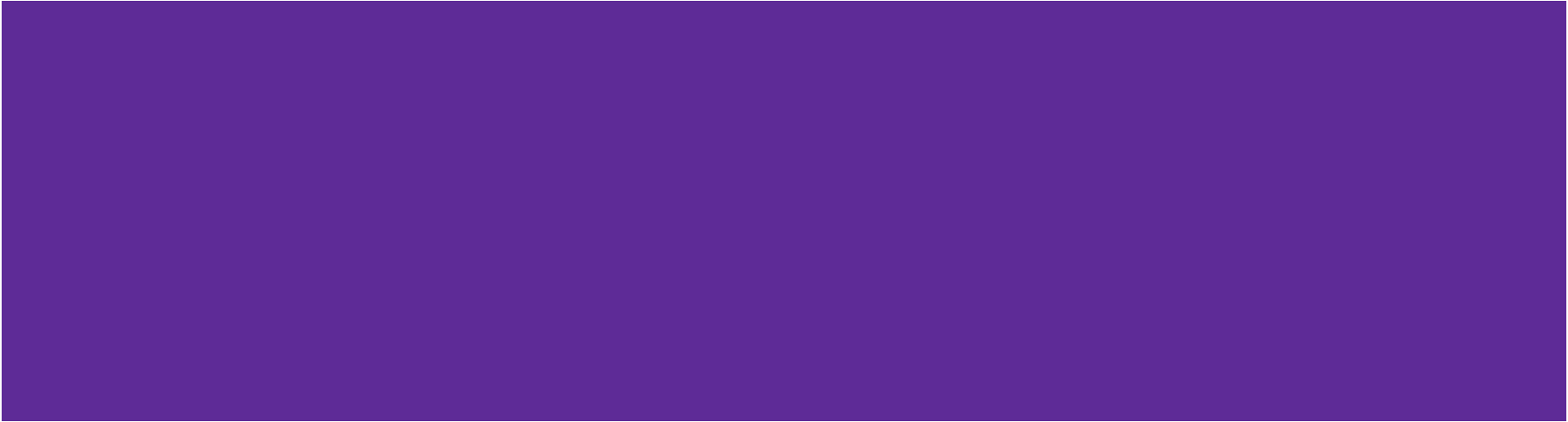


Projeto

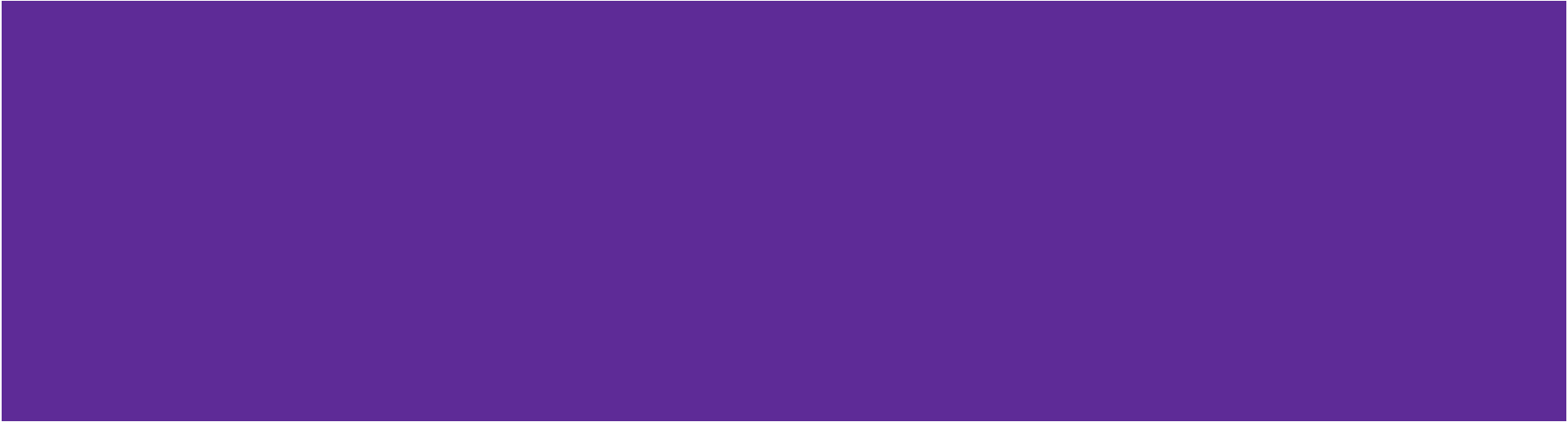
Editor Gráfico Vetorial – LP.II

Antônio Barreiros

<https://github.com/ahmbarreiros/LP2>



Classe Figure



Classe Figure

- Superclasse que possui os atributos comuns de figuras (posição, tamanho, contorno..)
- A classe Figure é abstrata, portanto só será usada caso uma classe herde seus atributos e métodos
- Métodos abstratos que devem ser implementados individualmente em cada classe herdeira

```
public abstract class Figure {  
    public int x, y;  
    public int w, h;  
    public float border = 2.0f;  
    public int contornoRGBIndex;  
    public Color contornoRGB = Color.BLACK;  
    public static ArrayList<Color> colors = new ArrayList<Color>(Arrays.asList(  
  
    public Figure (int x, int y, int w, int h, int contornoRGBIndex) {  
        this.x = x;  
        this.y = y;  
        this.w = w;  
        this.h = h;  
        this.contornoRGBIndex = contornoRGBIndex;  
    }  
}
```

```
    public abstract void changeFillD();  
    public abstract void changeFillU();  
  
    public abstract void paint (Graphics g);  
}
```

Figuras

Figuras que herdam da superclasse Figure

*As figuras e sua superclasse são empacotadas no pacote figures, declarado como:
package figures;

- Retângulo
- Ellipse
- Arco
- Reta

Classe Retângulo

1. Construção da classe Rect, herdando Figure.

2. Implementação de método abstrato herdado de Figure.

```
6 public class Rect extends Figure {  
7     private int fillRGBIndex = 0;  
8     private boolean paintBG = false;  
9  
10    public Rect (int x, int y, int w, int h, int contornoRGBIndex) {  
11        super(x,y, w,h, contornoRGBIndex);  
12    }  
13 }
```

```
public void paint (Graphics g) {  
    Graphics2D g2d = (Graphics2D) g;  
    g2d.setColor(this.colors.get(contornoRGBIndex));  
    g2d.setStroke(new BasicStroke(this.border));  
    g2d.drawRect(this.x,this.y, this.w,this.h);  
    if(this.paintBG) {  
        g2d.setColor(this.colors.get(fillRGBIndex));  
        g2d.fillRect(this.x+(int)(Math.ceil(this.border/2)),  
                     this.y+(int)(Math.ceil(this.border/2)),  
                     this.w-(int)this.border,  
                     this.h-(int)this.border);  
    }  
    g2d.setStroke(new BasicStroke(2.0f));  
    g2d.setColor(Color.BLACK);  
}
```

Aplicação

Criação de figuras e foco

```
this.addKeyListener (  
    new KeyAdapter() {  
        public void keyPressed (KeyEvent evt) {  
            if (evt.getKeyChar() == 'r' || evt.getKeyChar() == 'R') {  
                Rect rect = new Rect(mousePosX-25, mousePosY-25, 50, 50, 0);  
                figs.add(rect);  
                focus = rect;  
                repaint(); // outer.repaint()  
            }  
        }  
    }  
);
```

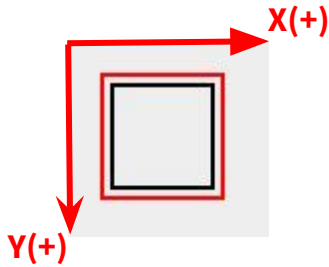
Evento mousePressed

```
    }  
    focus = null;  
    for (Figure fig: figs) {  
        if ((fig.x <= evt.getX() && (fig.x + fig.w) >= evt.getX()) && (fig.y <= evt.getY() && (fig.y + fig.h) >= evt.getY())) {  
            focus = fig;  
        }  
    }  
}
```

```
public void paint (Graphics g) {  
    super.paint(g);  
    Graphics2D g2d = (Graphics2D) g;  
  
    for (Figure fig: this.figs) {  
        fig.paint(g);  
    }  
    if (focus != null) {  
        g2d.setColor(new Color(255,0,0));  
        g2d.drawRect(focus.x-3-(int)focus.border, focus.y-3-(int)focus.border, focus.w+6+(int)(focus.border*2), focus.h+6+(int)(focus.border*2));  
        g2d.setColor(new Color(0,0,0));  
    }  
}
```

Lógica de redimensionamiento

```
if(
    evt.getX() >= focus.x-6-(int)focus.border && evt.getX() <= focus.x+6+(int)focus.border &&
    evt.getY() >= focus.y-6-(int)focus.border && evt.getY() <= focus.y+6+(int)focus.border
    ) {
    drag = "NW";
}
```



```
this.addMouseMotionListener(new MouseMotionAdapter() {
    public void mouseDragged(MouseEvent evt) {
        mousePosX = evt.getX();
        mousePosY = evt.getY();
        if (focus != null && drag != "") {
            switch(drag) {
                case "NW":
                    focus.transformNW(mousePosX - mousePosXPressed, mousePosY - mousePosYPressed);
                    break;
                case "NE":
                    focus.transformNE(mousePosX - mousePosXPressed, mousePosY - mousePosYPressed);
                    break;
                case "SW":
                    focus.transformSW(mousePosX - mousePosXPressed, mousePosY - mousePosYPressed);
                    break;
                case "SE":
                    focus.transformSE(mousePosX - mousePosXPressed, mousePosY - mousePosYPressed);
                    break;
            }
        }
    }
});
```

```
public void transformNW(int dw, int dh) {
    if(this.w + (-1*dw) <= 30) {
        this.w = 30;
    } else {
        this.w += (-1*dw);
        this.x += dw;
    }
    if(this.h + (-1*dh) <= 30) {
        this.h = 30;
    } else {
        this.h += (-1*dh);
        this.y += dh;
    }
}
```

Outros KeyEvents

Deletar figura e mudar cor de contorno

```
if (evt.getKeyCode() == KeyEvent.VK_DELETE) {  
    if (focus != null) {  
        figs.remove(focus);  
        focus = null;  
        repaint();  
    }  
}  
if (evt.getKeyCode() == KeyEvent.VK_RIGHT) {  
    if (focus != null) {  
        focus.changeBorderR();  
        repaint();  
    }  
}
```

Mudar cor de fundo e tamanho do contorno

```
if (evt.getKeyCode() == KeyEvent.VK_DOWN) {  
    if (focus != null) {  
        focus.changeFillD();  
        repaint();  
    }  
}  
if (evt.getKeyChar() == '=' || evt.getKeyChar() == '+') {  
    if (focus != null && focus.border <= 20.0f) {  
        focus.border += 2.0f;  
        repaint();  
    }  
}
```

Implementações

Alterar cor de contorno

```
public void changeBorderR() {
    this.contornoRGBIndex = (contornoRGBIndex+1) % colors.size();
}
public void changeBorderL() {
    if(contornoRGBIndex>0) {
        this.contornoRGBIndex = (contornoRGBIndex-1) % colors.size();
    }else{
        this.contornoRGBIndex = colors.size()-1;
    }
}
```

Desenho do contorno

```
g2d.setColor(this.colors.get(contornoRGBIndex));
g2d.setStroke(new BasicStroke(this.border));
g2d.draw(new Ellipse2D.Double(this.x,this.y, this.w,this.h));
```

Array de cores

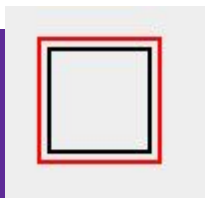
```
> colors = new ArrayList<Color>({
    Arrays.asList(
        Color.BLACK,
        Color.BLUE,
        Color.GREEN,
        Color.RED,
        Color.YELLOW,
        Color.ORANGE,
        Color.PINK,
        Color.WHITE
    )
});
```

Alterar cor de fundo

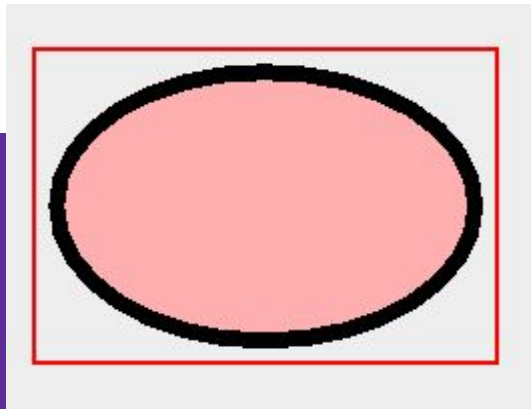
```
public void changeFillD() {
    this.paintBG = true;
    this.fillRGBIndex = (fillRGBIndex+1) % colors.size();
}
public void changeFillU() {
    this.paintBG = true;
    if(this.fillRGBIndex>0) {
        this.fillRGBIndex = (this.fillRGBIndex-1) % colors.size();
    }else{
        this.fillRGBIndex = colors.size()-1;
    }
}
```

Exemplos

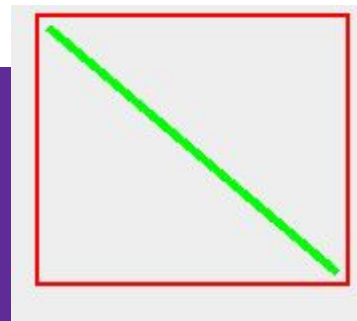
Retângulo em foco



Ellipse com cor de fundo e tamanho de borda alterada



Reta com cor e tamanho de contorno alterada



Demo